TALLER 2 — ARQUITECTURA EN CAPAS

Universidad Popular del Cesar · Programación III

Plan de Trabajo TransCesar

Sistema de Gestión de Tickets Intermunicipales · TransCesar S.A.S.

PERÍODO	TOTAL COMMITS	INTEGRANTES	DOCENTE
7 – 21 de marzo, 2026	37 commits	3 desarrolladores	Ing. Esp. Alfredo Bautista

01 Información General

PROYECTO
Sistema de Gestión de Tickets Intermunicipales

EMPRESA
TransCesar S.A.S.

ASIGNATURA
Programación III — Arquitectura en Capas

DOCENTE
Ing. Esp. Alfredo Bautista · adbautista@unicesar.edu.co

TECNOLOGÍA
Java SE — model / dao / service / view

PERÍODO DE DESARROLLO
7 de marzo al 21 de marzo de 2026

REPOSITORIO GITHUB
https://github.com/Hellum-d/TransCesar-Tickets

02 Equipo de Desarrollo

LÍDER DE DESARROLLO

Hellum-d

23

COMMITS REALIZADOS

Estructura base, interfaces, capa view, persistencia DAO, sistema de reservas, merges e integración de ramas.

DESARROLLADOR 1

Cessxr474

9

COMMITS REALIZADOS

Jerarquía Vehiculo (Buseta, MicroBus, Bus), clase Ruta, VehiculoDAO y VehiculoService.

DESARROLLADOR 2

moisescarrilo

5

COMMITS REALIZADOS

Jerarquía Persona, Ticket, PersonaDAO, TicketDAO y TicketService con reglas de negocio avanzadas.

03 Cronograma Real de Desarrollo

MAR 7, 2026

Creación del repositorio y commit inicial

Hellum-d — Líder

~1h

MAR 10, 2026

Interfaces Imprimible y Calculable — Main.java — MenuPrincipal.java

Hellum-d — Líder

~3h

MAR 11, 2026

Clase abstracta Vehiculo con atributos base

Cessxr474 — Desarrollador 1

~2h

MAR 12, 2026

Clases Buseta, MicroBus y Bus con imprimirDetalle()

Cessxr474 — Desarrollador 1

~3h

MAR 14, 2026

Clase Ruta, integración en Vehiculo, modelo Pasajero con fecha de nacimiento

Cessxr474 — Desarrollador 1

~4h

MAR 14, 2026

Jerarquía Persona, Ticket, PersonaDAO, TicketDAO y TicketService con reglas de negocio

moisescarrilo — Desarrollador 2

~6h

MAR 14, 2026

Merges, resolución de conflictos, clases vacías en capas dao y service

Hellum-d — Líder

~2h

MAR 16, 2026

Submenús y módulo de reportes completo en MenuPrincipal

Hellum-d — Líder

~3h

MAR 19, 2026

Persistencia en todos los DAO, sistema de reservas completo, validaciones y correcciones finales

Hellum-d — Líder

~8h

MAR 21, 2026

fix: corregir Main.java para arrancar el sistema con menuPrincipal

Hellum-d — Líder

~15min

MAR 21, 2026

Corrección de Main.java para arrancar el sistema con menuPrincipal

Hellum-d — Líder

~15min

04 Estrategia de Ramas en GitHub

🌿 **`main`**

Rama principal. Solo el Líder realiza merges aquí.

🚗 **`feature/vehiculos`**

Jerarquía de vehículos, VehiculoDAO y VehiculoService.

👤 **`feature/personas-tickets`**

Jerarquía Persona, Ticket, TicketDAO y TicketService.

🗺️ **`feature/gestion-rutas`**

Requerimiento 1: clase Ruta y validación automática de edad.

⚙️ **`feature/reglas-negocio`**

Requerimiento 2: límite de tickets por día y tarifa en festivos.

📊 **`feature/reportes-filtros`**

Requerimiento 3: módulo de reportes con filtros.

🎫 **`feature/sistema-reservas`**

Requerimiento 4: sistema de reservas completo (model, dao, service, view).

05 Historial Completo de Commits

FECHA	MENSAJE DE COMMIT	AUTOR	ROL
Mar 7	`Initial commit`	Hellum-d	Líder
Mar 10	`feat: agregar interfaces Imprimible y Calculable`	Hellum-d	Líder
Mar 10	`feat: agregar Main.java y MenuPrincipal.java`	Hellum-d	Líder
Mar 11	`feat: crear clase abstracta Vehiculo con atributos base`	Cessxr474	Dev 1
Mar 12	`feat: implementar clase Buseta con imprimirDetalle`	Cessxr474	Dev 1
Mar 12	`feat: implementar clase MicroBus con capacidad de 25`	Cessxr474	Dev 1

FECHA	MENSAJE DE COMMIT	AUTOR	ROL
Mar 12	feat: Se crea la clase Bus con su capacidad maxima de 45 personas	Cessxr474	Dev 1
Mar 14	feat: crear clase Ruta con atributos detallados	Cessxr474	Dev 1
Mar 14	feat: integrar objeto Ruta en la jerarquia de vehiculos	Cessxr474	Dev 1
Mar 14	feat: agregar modelo Pasajero con fecha de nacimiento	Cessxr474	Dev 1
Mar 14	feat: crear clase Ruta e integrar pruebas en el Main	Cessxr474	Dev 1
Mar 14	feat: completar arquitectura DAO y Service	Cessxr474	Dev 1
Mar 14	avance jerarquia personas y ticket	moisescarrilo	Dev 2
Mar 14	implementacion PersonaDAO y TicketDAO	moisescarrilo	Dev 2
Mar 14	correccion clase ticket y calculo total	moisescarrilo	Dev 2
Mar 14	implementacion de estadisticas en ticketService	moisescarrilo	Dev 2
Mar 14	implementacion de logica de negocio ticketService	moisescarrilo	Dev 2
Mar 14	merge: resolver conflictos feature/gestion-rutas	Hellum-d	Líder
Mar 14	chore: agregar clases vacias en capas dao y service	Hellum-d	Líder
Mar 14	Merge remote-tracking branch 'origin/feature/gestion-rutas'	Hellum-d	Líder
Mar 16	fix: quitar getTarifaBase de VehiculoDAO	Hellum-d	Líder
Mar 16	feat: agregar submenus y modulo de reportes en MenuPrincipal	Hellum-d	Líder
Mar 19	feat: agregar atributo fechaCompra a clase Ticket	Hellum-d	Líder
Mar 19	fix: corregir validacion limite tickets por dia con fechaCompra	Hellum-d	Líder
Mar 19	feat: agregar persistencia en archivo a VehiculoDAO y validaciones	Hellum-d	Líder
Mar 19	Merge remote-tracking branch 'origin/feature/vehiculos'	Hellum-d	Líder
Mar 19	feat: agregar persistencia en ConductorDAO, PasajeroDAO, TicketDAO y PersonaService	Hellum-d	Líder

FECHA	MENSAJE DE COMMIT	AUTOR	ROL
Mar 19	`chore: crear clases vacias Reserva, ReservaDAO y ReservaService`	Hellum-d	Líder
Mar 19	`feat: agregar persistencia en ReservaDAO con archivo reservas.txt`	Hellum-d	Líder
Mar 19	`feat: agregar logica de negocio en ReservaService`	Hellum-d	Líder
Mar 19	`feat: agregar modulo de reservas en MenuPrincipal`	Hellum-d	Líder
Mar 19	`fix: corregir clases Reserva y ReservaService que no se guardaron`	Hellum-d	Líder
Mar 19	`fix: VehiculoDao no se guardo correctamente`	Hellum-d	Líder
Mar 19	`fix: Ticket.Dao no se guardo correctamente`	Hellum-d	Líder
Mar 19	`feat: agregar validaciones de placa y licencia en VehiculoService`	Hellum-d	Líder
Mar 21	`fix: corregir Main.java para arrancar el sistema con menuPrincipal`	Hellum-d	Líder
Mar 21	`fix: corregir Main.java para arrancar el sistema con menuPrincipal`	Hellum-d	Líder

06 Criterios de Evaluación

CRITERIO 1 — 10

Herencia e Interfaces

Clases abstractas Vehiculo y Persona definidas con sus subclases. Interfaces Imprimible y Calculable implementadas. Polimorfismo con calcularDescuento() funciona correctamente.

CRITERIO 2 — 8

Persistencia en Archivos

Cada entidad tiene su propio archivo .txt. Datos guardados al registrar y cargados automáticamente al iniciar. Delimitador ; usado consistentemente.

CRITERIO 3 — 7

Código Funcional

Sistema compila y ejecuta sin errores. Registro de vehículos, personas, venta de tickets, consultas,

CRITERIO 4 — 5

Diagrama de Clases

Diagrama refleja todas las clases con atributos, métodos, relaciones de herencia, implementación

estadísticas y reservas funcionando. Validaciones del service implementadas.

de interfaces y asociaciones. Consistente con el código.

CRITERIO 5 10

Git y Commits

Los tres integrantes tienen commits. Se usaron ramas individuales y merges a main. Mensajes siguen convenciones de Conventional Commits.

CRITERIO 6 4

Arquitectura en Capas

Los cuatro paquetes model, dao, service y view presentes. Flujo view → service → dao → model respetado. Ninguna capa se salta a otra.

CRITERIO 7 6

Plan de Trabajo

Plan describe actividades de cada integrante con claridad, tiempos estimados y coherente con el historial de commits del repositorio.

Puntaje máximo total **50 pts**

07 Entregables

1. Repositorio público en GitHub con código Java funcional
   ✓ Completado
2. Diagrama de clases completo con todas las capas
   ✓ Completado
3. Historial de commits con trabajo de los 3 integrantes
   ✓ Completado
4. Plan de trabajo con especificidad de actividades
   ✓ Completado